

Snaplet

Complete SaaS product generationShared by Shrawan

This is a copy of a chat between Claude and Shrawan. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

You are a senior full-stack engineer, SaaS architect, and product founder. Your task is to generate a COMPLETE, PRODUCTION-READY SaaS product. This is NOT a demo, NOT a tutorial, and NOT partial code. You must generate: - frontend - backend - database schema - authentication - payments**pasted**

4:37 pm

I'll generate a complete, production-ready SaaS product for you. This will be comprehensive and fully functional.



COMPLETE SAAS PRODUCT: MEMVAULT

Project Structure

```
memvault/  
├── backend/  
│   ├── config/  
│   │   ├── database.js  
│   │   ├── cloudinary.js  
│   │   └── passport.js  
│   ├── middleware/  
│   │   ├── auth.js  
│   │   └── upload.js  
│   ├── routes/  
│   └── auth.js
```

give all files here in chat, dont use docker local fine

4:37 pm



COMPLETE SAAS PRODUCT: MEMVAULT

I'll provide ALL files for a complete, production-ready SaaS application.

PROJECT STRUCTURE

```
memvault/  
├── backend/  
│   ├── config/  
│   │   ├── database.js  
│   │   ├── cloudinary.js  
│   │   └── passport.js  
│   ├── middleware/  
│   │   ├── auth.js  
│   │   └── upload.js  
│   ├── routes/  
│   │   ├── auth.js  
│   │   ├── files.js  
│   │   ├── search.js  
│   │   └── payment.js  
│   ├── services/  
│   │   ├── ocr.js  
│   │   ├── documentParser.js  
│   │   ├── embedding.js  
│   │   └── keywordExtractor.js  
│   ├── controllers/  
│   │   ├── authController.js  
│   │   ├── fileController.js  
│   │   ├── searchController.js  
│   │   └── paymentController.js  
│   ├── models/  
│   │   └── schema.sql  
│   ├── .env.example  
│   ├── server.js  
│   └── package.json  
├── frontend/  
│   ├── public/  
│   │   └── index.html  
│   ├── src/  
│   │   ├── components/  
│   │   │   ├── Navbar.js  
│   │   │   ├── LandingPage.js  
│   │   │   ├── Login.js  
│   │   │   ├── Signup.js  
│   │   │   ├── Dashboard.js  
│   │   │   ├── UploadSection.js  
│   │   │   ├── FileCard.js  
│   │   │   ├── SearchBar.js  
│   │   │   ├── PaywallModal.js  
│   │   │   └── PrivateRoute.js  
│   │   ├── services/  
│   │   │   └── api.js  
│   │   ├── App.js  
│   │   ├── App.css  
│   │   └── index.js  
│   ├── package.json  
│   ├── .env.example  
└── README.md
```



DATABASE SCHEMA

backend/models/schema.sql

sql

- - Enable pgvector extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

- Users table

```
CREATE TABLE users ( id SERIAL PRIMARY KEY, email VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255), google_id VARCHAR(255) UNIQUE, country VARCHAR(10) DEFAULT 'IN', is_paid BOOLEAN DEFAULT FALSE, free_uploads_used INTEGER DEFAULT 0, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

- Files table

```
CREATE TABLE files ( id SERIAL PRIMARY KEY, user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, file_url TEXT NOT NULL, file_type VARCHAR(50) NOT NULL, file_name VARCHAR(255) NOT NULL, extracted_text TEXT, embedding vector(1536), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

- Payments table

```
CREATE TABLE payments ( id SERIAL PRIMARY KEY, user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, razorpay_order_id VARCHAR(255), razorpay_payment_id VARCHAR(255), razorpay_signature VARCHAR(255), amount DECIMAL(10, 2) NOT NULL, currency VARCHAR(10) NOT NULL, status VARCHAR(50) DEFAULT 'pending', created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

- Indexes for performance

```
CREATE INDEX idx_files_user_id ON files(user_id);
CREATE INDEX idx_files_created_at ON files(created_at DESC);
CREATE INDEX idx_payments_user_id ON payments(user_id);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_google_id ON users(google_id);
```

- Vector similarity search index

```
CREATE INDEX ON files USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```



BACKEND FILES

backend/package.json

json

```
{
  "name": "memvault-backend",
  "version": "1.0.0",
  "description": "MemVault Backend API",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
```

```

"express": "^4.18.2",
"pg": "^8.11.3",
"pgvector": "^0.1.8",
"bcryptjs": "^2.4.3",
"jsonwebtoken": "^9.0.2",
"passport": "^0.7.0",
"passport-google-oauth20": "^2.0.0",
"passport-jwt": "^4.0.1",
"dotenv": "^16.3.1",
"cors": "^2.8.5",
"multer": "^1.4.5-lts.1",
"cloudinary": "^1.41.0",
"tesseract.js": "^5.0.4",
"axios": "^1.6.2",
"openai": "^4.20.1",
"razorpay": "^2.9.2",
"crypto": "^1.0.1",
"express-validator": "^7.0.1",
"helmet": "^7.1.0",
"express-rate-limit": "^7.1.5",
"pdf-parse": "^1.1.1",
"mammoth": "^1.6.0"
},
"devDependencies": {
  "nodemon": "^3.0.2"
}
}

```

backend/.env.example

env

```

# Server
PORT=5000
NODE_ENV=production
FRONTEND_URL=http://localhost:3000

# Database
DATABASE_URL=postgresql://username:password@localhost:5432/memvault

# JWT
JWT_SECRET=your_super_secret_jwt_key_min_32_characters_long_random_string

# Google OAuth
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/auth/google/callback

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# OpenAI
OPENAI_API_KEY=your_openai_api_key

```

```
# Razorpay
RAZORPAY_KEY_ID=your_razorpay_key_id
RAZORPAY_KEY_SECRET=your_razorpay_key_secret
```

```
# Pricing
INDIA_PRICE=9900
INDIA_CURRENCY=INR
INTERNATIONAL_PRICE=1500
INTERNATIONAL_CURRENCY=USD
```

```
# Apache Tika Server (run separately)
TIKA_SERVER_URL=http://localhost:9998
```

backend/config/database.js

javascript

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
});

pool.on('connect', () => {
  console.log('✅ Database connected successfully');
});

pool.on('error', (err) => {
  console.error('❌ Unexpected database error:', err);
  process.exit(-1);
});

module.exports = pool;
```

backend/config/cloudinary.js

javascript

```
const cloudinary = require('cloudinary').v2;
require('dotenv').config();

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

module.exports = cloudinary;
```

backend/config/passport.js

javascript

```

const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const JwtStrategy = require('passport-jwt').Strategy;
const ExtractJwt = require('passport-jwt').ExtractJwt;
const pool = require('./database');
require('dotenv').config();

// JWT Strategy
const jwtOptions = {
  jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  secretOrKey: process.env.JWT_SECRET
};

passport.use(new JwtStrategy(jwtOptions, async (payload, done) => {
  try {
    const result = await pool.query('SELECT * FROM users WHERE id = $1', [payload.id]);
    if (result.rows.length > 0) {
      return done(null, result.rows[0]);
    }
    return done(null, false);
  } catch (error) {
    return done(error, false);
  }
}));

// Google OAuth Strategy
passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.GOOGLE_CALLBACK_URL
}, async (accessToken, refreshToken, profile, done) => {
  try {
    const email = profile.emails[0].value;
    const googleId = profile.id;

    // Check if user exists
    let result = await pool.query('SELECT * FROM users WHERE google_id = $1', [googleId]);

    if (result.rows.length > 0) {
      return done(null, result.rows[0]);
    }

    // Check if email exists
    result = await pool.query('SELECT * FROM users WHERE email = $1', [email]);

    if (result.rows.length > 0) {
      // Link Google account to existing user
      await pool.query('UPDATE users SET google_id = $1 WHERE email = $2', [googleId, email]);
      return done(null, result.rows[0]);
    }

    // Create new user
    result = await pool.query(
      'INSERT INTO users (email, google_id, country) VALUES ($1, $2, $3) RETURNING *',
      [email, googleId, 'IN']
    );
  }
});

```

```

    return done(null, result.rows[0]);
  } catch (error) {
    return done(error, false);
  }
});

```

```
module.exports = passport;
```

backend/middleware/auth.js

javascript

```

const jwt = require('jsonwebtoken');
const pool = require('../config/database');

const authenticate = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Authentication required' });
    }

    const token = authHeader.substring(7);
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const result = await pool.query('SELECT * FROM users WHERE id = $1', [decoded.id]);

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'User not found' });
    }

    req.user = result.rows[0];
    next();
  } catch (error) {
    if (error.name === 'JsonWebTokenError') {
      return res.status(401).json({ error: 'Invalid token' });
    }
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expired' });
    }
    return res.status(500).json({ error: 'Authentication error' });
  }
};

const checkUploadLimit = async (req, res, next) => {
  try {
    const userId = req.user.id;

    const result = await pool.query(
      'SELECT is_paid, free_uploads_used FROM users WHERE id = $1',
      [userId]
    );

    const user = result.rows[0];

```

```

    if (user.is_paid) {
      return next();
    }

    if (user.free_uploads_used >= 5) {
      return res.status(403).json({
        error: 'Free upload limit reached',
        requiresPayment: true,
        message: 'You have used all 5 free uploads. Please upgrade to continue.'
      });
    }

    next();
  } catch (error) {
    console.error('Error checking upload limit:', error);
    return res.status(500).json({ error: 'Error checking upload limit' });
  }
};

module.exports = { authenticate, checkUploadLimit };

```

backend/middleware/upload.js

javascript

```

const multer = require('multer');

const storage = multer.memoryStorage();

const fileFilter = (req, file, cb) => {
  const allowedTypes = [
    'image/jpeg',
    'image/png',
    'image/jpg',
    'application/pdf',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
  ];

  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type. Only JPG, PNG, PDF, and DOCX files are allowed.'), false);
  }
};

const upload = multer({
  storage: storage,
  fileFilter: fileFilter,
  limits: {
    fileSize: 10 * 1024 * 1024 // 10MB limit
  }
});

module.exports = upload;

```

backend/services/ocr.js

javascript

```
const Tesseract = require('tesseract.js');

const extractTextFromImage = async (imageBuffer) => {
  try {
    console.log('Starting OCR extraction...');

    const { data: { text } } = await Tesseract.recognize(
      imageBuffer,
      'eng',
      {
        logger: info => console.log(info)
      }
    );

    const cleanedText = text.trim().replace(/\s+/g, ' ');
    console.log('OCR extraction completed');

    return cleanedText;
  } catch (error) {
    console.error('OCR extraction error:', error);
    throw new Error('Failed to extract text from image');
  }
};

module.exports = { extractTextFromImage };
```

backend/services/documentParser.js

javascript

```
const axios = require('axios');
const pdfParse = require('pdf-parse');
const mammoth = require('mammoth');

const extractTextFromPDF = async (fileBuffer) => {
  try {
    console.log('Extracting text from PDF...');
    const data = await pdfParse(fileBuffer);
    const cleanedText = data.text.trim().replace(/\s+/g, ' ');
    console.log('PDF extraction completed');
    return cleanedText;
  } catch (error) {
    console.error('PDF extraction error:', error);
    throw new Error('Failed to extract text from PDF');
  }
};

const extractTextFromDOCX = async (fileBuffer) => {
  try {
    console.log('Extracting text from DOCX...');
    const result = await mammoth.extractRawText({ buffer: fileBuffer });
    const cleanedText = result.value.trim().replace(/\s+/g, ' ');
  }
};
```

```

    console.log('DOCX extraction completed');
    return cleanedText;
  } catch (error) {
    console.error('DOCX extraction error:', error);
    throw new Error('Failed to extract text from DOCX');
  }
};

// Alternative: Apache Tika (requires separate Tika server running)
const extractTextWithTika = async (fileBuffer, mimeType) => {
  try {
    console.log('Extracting text with Apache Tika...');

    const response = await axios.put(
      `${process.env.TIKA_SERVER_URL}/tika`,
      fileBuffer,
      {
        headers: {
          'Content-Type': mimeType,
          'Accept': 'text/plain'
        },
        maxLength: Infinity,
        bodyLength: Infinity
      }
    );

    const cleanedText = response.data.trim().replace(/\s+/g, ' ');
    console.log('Tika extraction completed');
    return cleanedText;
  } catch (error) {
    console.error('Tika extraction error:', error);
    throw new Error('Failed to extract text with Tika');
  }
};

module.exports = {
  extractTextFromPDF,
  extractTextFromDOCX,
  extractTextWithTika
};

```

backend/services/embedding.js

javascript

```

const OpenAI = require('openai');
require('dotenv').config();

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
});

const generateEmbedding = async (text) => {
  try {
    if (!text || text.trim().length === 0) {
      throw new Error('Text is empty');
    }
  }

```

```

    }

    console.log('Generating embedding...');

    // Truncate text if too long (max ~8000 tokens for text-embedding-ada-002)
    const maxLength = 8000;
    const truncatedText = text.length > maxLength ? text.substring(0, maxLength) : text;

    const response = await openai.embeddings.create({
      model: 'text-embedding-ada-002',
      input: truncatedText
    });

    console.log('Embedding generated successfully');
    return response.data[0].embedding;
  } catch (error) {
    console.error('Embedding generation error:', error);
    throw new Error('Failed to generate embedding');
  }
};

module.exports = { generateEmbedding };

```

backend/services/keywordExtractor.js

javascript

```

const extractKeywords = (text) => {
  if (!text || text.trim().length === 0) {
    return [];
  }

  // Common stop words to filter out
  const stopWords = new Set([
    'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were',
    'been', 'be', 'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
    'could', 'should', 'may', 'might', 'must', 'can', 'of', 'to', 'in', 'for',
    'with', 'by', 'from', 'about', 'into', 'through', 'during', 'before',
    'after', 'above', 'below', 'between', 'under', 'again', 'further', 'then',
    'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'both',
    'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor',
    'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 'and', 'but',
    'or', 'if', 'because', 'until', 'while', 'this', 'that', 'these', 'those'
  ]);

  const words = text.toLowerCase().match(/\b[\w]+\b/g) || [];
  const numbers = text.match(/\d+/g) || [];

  // Extract capitalized terms from original text (potential proper nouns)
  const capitalizedTerms = text.match(/\b[A-Z][a-z]+\b/g) || [];

  // Filter and count word frequency
  const wordFrequency = {};
  words.forEach(word => {
    if (word.length > 3 && !stopWords.has(word)) {
      wordFrequency[word] = (wordFrequency[word] || 0) + 1;
    }
  });
  capitalizedTerms.forEach(word => {
    wordFrequency[word] = (wordFrequency[word] || 0) + 1;
  });
  numbers.forEach(word => {
    wordFrequency[word] = (wordFrequency[word] || 0) + 1;
  });

  return Object.keys(wordFrequency).map(word => {
    return { word, frequency: wordFrequency[word] };
  });
};

```

```

    }
  });

  // Sort by frequency
  const sortedWords = Object.entries(wordFrequency)
    .sort((a, b) => b[1] - a[1])
    .map(([word]) => word)
    .slice(0, 5);

  // Combine results
  const keywords = [
    ...new Set([
      ...capitalizedTerms.slice(0, 3).map(w => w.toLowerCase()),
      ...sortedWords,
      ...numbers.slice(0, 3)
    ])
  ];

  return keywords.slice(0, 8);
};

module.exports = { extractKeywords };

```

backend/controllers/authController.js

javascript

```

const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../config/database');
const { validationResult } = require('express-validator');

const generateToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: '30d' });
};

const signup = async (req, res) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const { email, password, country } = req.body;

    // Check if user exists
    const existingUser = await pool.query('SELECT * FROM users WHERE email = $1', [email]);

    if (existingUser.rows.length > 0) {
      return res.status(400).json({ error: 'Email already registered' });
    }

    // Hash password
    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

```

```

    // Create user
    const result = await pool.query(
      'INSERT INTO users (email, password_hash, country) VALUES ($1, $2, $3) RETURNING id, email, country, is_paid, free_uploads_used',
      [email, passwordHash, country || 'IN']
    );

    const user = result.rows[0];
    const token = generateToken(user.id);

    res.status(201).json({
      token,
      user: {
        id: user.id,
        email: user.email,
        country: user.country,
        is_paid: user.is_paid,
        free_uploads_used: user.free_uploads_used
      }
    });
  } catch (error) {
    console.error('Signup error:', error);
    res.status(500).json({ error: 'Server error during signup' });
  }
};

const login = async (req, res) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const { email, password } = req.body;

    // Find user
    const result = await pool.query('SELECT * FROM users WHERE email = $1', [email]);

    if (result.rows.length === 0) {
      return res.status(400).json({ error: 'Invalid credentials' });
    }

    const user = result.rows[0];

    // Check password
    if (!user.password_hash) {
      return res.status(400).json({ error: 'Please login with Google' });
    }

    const isMatch = await bcrypt.compare(password, user.password_hash);

    if (!isMatch) {
      return res.status(400).json({ error: 'Invalid credentials' });
    }

    const token = generateToken(user.id);

```

```

    res.json({
      token,
      user: {
        id: user.id,
        email: user.email,
        country: user.country,
        is_paid: user.is_paid,
        free_uploads_used: user.free_uploads_used
      }
    });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ error: 'Server error during login' });
  }
};

const googleAuthCallback = async (req, res) => {
  try {
    const token = generateToken(req.user.id);

    // Redirect to frontend with token
    res.redirect(`${process.env.FRONTEND_URL}/auth/google/callback?token=${token}`);
  } catch (error) {
    console.error('Google auth callback error:', error);
    res.redirect(`${process.env.FRONTEND_URL}/login?error=auth_failed`);
  }
};

const getCurrentUser = async (req, res) => {
  try {
    res.json({
      user: {
        id: req.user.id,
        email: req.user.email,
        country: req.user.country,
        is_paid: req.user.is_paid,
        free_uploads_used: req.user.free_uploads_used
      }
    });
  } catch (error) {
    console.error('Get current user error:', error);
    res.status(500).json({ error: 'Server error' });
  }
};

module.exports = {
  signup,
  login,
  googleAuthCallback,
  getCurrentUser
};

```

backend/controllers/fileController.js

javascript

```
const pool = require('../config/database');
const cloudinary = require('../config/cloudinary');
const { extractTextFromImage } = require('../services/ocr');
const { extractTextFromPDF, extractTextFromDOCX } = require('../services/documentParser');
const { generateEmbedding } = require('../services/embedding');
const { extractKeywords } = require('../services/keywordExtractor');
const { Readable } = require('stream');
```

```
const uploadFile = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'No file uploaded' });
    }
  }
```

```
    const userId = req.user.id;
    const file = req.file;
    const fileType = file.mimetype;
```

```
    console.log(`Processing file: ${file.originalname}, Type: ${fileType}`);
```

```
    // Upload to Cloudinary
    let cloudinaryResult;

    if (fileType.startsWith('image/')) {
      // Upload image normally
      cloudinaryResult = await new Promise((resolve, reject) => {
        const uploadStream = cloudinary.uploader.upload_stream(
          { folder: 'memvault', resource_type: 'image' },
          (error, result) => {
            if (error) reject(error);
            else resolve(result);
          }
        );
        Readable.from(file.buffer).pipe(uploadStream);
      });
    } else {
      // Upload PDF/DOCX as raw
      cloudinaryResult = await new Promise((resolve, reject) => {
        const uploadStream = cloudinary.uploader.upload_stream(
          { folder: 'memvault', resource_type: 'raw' },
          (error, result) => {
            if (error) reject(error);
            else resolve(result);
          }
        );
        Readable.from(file.buffer).pipe(uploadStream);
      });
    }
  }
```

```
  console.log('File uploaded to Cloudinary');
```

```
  // Extract text based on file type
  let extractedText = '';
```

```

    if (fileType.startsWith('image/')) {
      extractedText = await extractTextFromImage(file.buffer);
    } else if (fileType === 'application/pdf') {
      extractedText = await extractTextFromPDF(file.buffer);
    } else if (fileType === 'application/vnd.openxmlformats-officedocument.wordprocessingml.document') {
      extractedText = await extractTextFromDOCX(file.buffer);
    }
  }

  if (!extractedText || extractedText.length < 10) {
    await cloudinary.uploader.destroy(cloudinaryResult.public_id, {
      resource_type: fileType.startsWith('image/') ? 'image' : 'raw'
    });
    return res.status(400).json({ error: 'No text could be extracted from the file' });
  }

  console.log(`Extracted text length: ${extractedText.length} characters`);

  // Generate embedding
  const embedding = await generateEmbedding(extractedText);

  // Extract keywords for display
  const keywords = extractKeywords(extractedText);

  // Save to database
  const result = await pool.query(
    `INSERT INTO files (user_id, file_url, file_type, file_name, extracted_text, embedding)
    VALUES ($1, $2, $3, $4, $5, $6::vector)
    RETURNING id, file_url, file_type, file_name, created_at`,
    [
      userId,
      cloudinaryResult.secure_url,
      fileType,
      file.originalName,
      extractedText,
      JSON.stringify(embedding)
    ]
  );

  // Increment free uploads counter
  if (!req.user.is_paid) {
    await pool.query(
      `UPDATE users SET free_uploads_used = free_uploads_used + 1 WHERE id = $1`,
      [userId]
    );
  }

  console.log('File saved to database successfully');

  res.status(201).json({
    message: 'File uploaded successfully',
    file: {
      ...result.rows[0],
      keywords: keywords
    }
  });
} catch (error) {

```



```

    console.error('Upload error:', error);
    res.status(500).json({ error: 'Failed to upload file', details: error.message });
  }
};

const getUserFiles = async (req, res) => {
  try {
    const userId = req.user.id;
    const { limit = 50, offset = 0 } = req.query;

    const result = await pool.query(
      `SELECT id, file_url, file_type, file_name, extracted_text, created_at
      FROM files
      WHERE user_id = $1
      ORDER BY created_at DESC
      LIMIT $2 OFFSET $3`,
      [userId, limit, offset]
    );

    // Extract keywords for each file
    const filesWithKeywords = result.rows.map(file => ({
      ...file,
      keywords: extractKeywords(file.extracted_text || ''),
      extracted_text: undefined // Don't send full text to frontend
    }));

    res.json({ files: filesWithKeywords });
  } catch (error) {
    console.error('Get files error:', error);
    res.status(500).json({ error: 'Failed to retrieve files' });
  }
};

const deleteFile = async (req, res) => {
  try {
    const userId = req.user.id;
    const fileId = req.params.id;

    // Get file details
    const fileResult = await pool.query(
      'SELECT file_url, file_type FROM files WHERE id = $1 AND user_id = $2',
      [fileId, userId]
    );

    if (fileResult.rows.length === 0) {
      return res.status(404).json({ error: 'File not found' });
    }

    const file = fileResult.rows[0];

    // Extract public_id from Cloudinary URL
    const urlParts = file.file_url.split('/');
    const fileName = urlParts[urlParts.length - 1];
    const publicId = `memvault/${fileName.split('.')[0]}`;

    // Delete from Cloudinary

```

```

try {
  await cloudinary.uploader.destroy(publicId, {
    resource_type: file.file_type.startsWith('image/') ? 'image' : 'raw'
  });
}

```

Continue

4:40 pm

```

} catch (cloudinaryError) {
  console.error('Cloudinary deletion error:', cloudinaryError);
}

// Delete from database
await pool.query('DELETE FROM files WHERE id = $1', [fileId]);

res.json({ message: 'File deleted successfully' });

} catch (error) {
  console.error('Delete file error:', error);
  res.status(500).json({ error: 'Failed to delete file' });
}
};

module.exports = {
  uploadFile,
  getUserFiles,
  deleteFile
};

```

```

### `backend/controllers/searchController.js`
```javascript
const pool = require('../config/database');
const { generateEmbedding } = require('../services/embedding');
const { extractKeywords } = require('../services/keywordExtractor');

const searchFiles = async (req, res) => {
 try {
 const userId = req.user.id;
 const { query, limit = 20 } = req.body;

 if (!query || query.trim().length === 0) {
 return res.status(400).json({ error: 'Search query is required' });
 }

 console.log(`Searching for: "${query}"`);

 // Generate embedding for search query
 const queryEmbedding = await generateEmbedding(query);

 // Perform vector similarity search
 const result = await pool.query(

```

```

`SELECT
 id,
 file_url,
 file_type,
 file_name,
 extracted_text,
 created_at,
 1 - (embedding ⇔ $1::vector) as similarity
FROM files
WHERE user_id = $2
ORDER BY embedding ⇔ $1::vector
LIMIT $3`,
[JSON.stringify(queryEmbedding), userId, limit]
);

// Filter results with reasonable similarity threshold
const relevantFiles = result.rows.filter(file ⇒ file.similarity > 0.5);

// Extract keywords and prepare response
const filesWithKeywords = relevantFiles.map(file ⇒ ({
 id: file.id,
 file_url: file.file_url,
 file_type: file.file_type,
 file_name: file.file_name,
 created_at: file.created_at,
 keywords: extractKeywords(file.extracted_text || ''),
 similarity: file.similarity,
 preview: file.extracted_text ? file.extracted_text.substring(0, 200) : ''
})));

console.log(`Found ${filesWithKeywords.length} relevant files`);

res.json({
 results: filesWithKeywords,
 count: filesWithKeywords.length
});
} catch (error) {
 console.error('Search error:', error);
 res.status(500).json({ error: 'Search failed', details: error.message });
}
};

module.exports = {
 searchFiles
};
'''

`backend/controllers/paymentController.js`
'''javascript
const Razorpay = require('razorpay');
const crypto = require('crypto');
const pool = require('../config/database');
require('dotenv').config();

const razorpay = new Razorpay({
 key_id: process.env.RAZORPAY_KEY_ID,

```

```

 key_secret: process.env.RAZORPAY_KEY_SECRET
 });

const createOrder = async (req, res) => {
 try {
 const userId = req.user.id;
 const userCountry = req.user.country;

 // Determine pricing based on country
 let amount, currency;
 if (userCountry === 'IN') {
 amount = parseInt(process.env.INDIA_PRICE); // Amount in paise (₹99 = 9900 paise)
 currency = process.env.INDIA_CURRENCY;
 } else {
 amount = parseInt(process.env.INTERNATIONAL_PRICE); // Amount in cents ($15 = 1500 cents)
 currency = process.env.INTERNATIONAL_CURRENCY;
 }

 // Create Razorpay order
 const order = await razorpay.orders.create({
 amount: amount,
 currency: currency,
 receipt: `receipt_${userId}_${Date.now()}`,
 notes: {
 user_id: userId
 }
 });

 // Save order to database
 await pool.query(
 'INSERT INTO payments (user_id, razorpay_order_id, amount, currency, status) VALUES ($1, $2, $3, $4, $5)',
 [userId, order.id, amount / 100, currency, 'pending']
);

 res.json({
 orderId: order.id,
 amount: amount,
 currency: currency,
 keyId: process.env.RAZORPAY_KEY_ID
 });
 } catch (error) {
 console.error('Create order error:', error);
 res.status(500).json({ error: 'Failed to create payment order' });
 }
};

const verifyPayment = async (req, res) => {
 try {
 const { razorpay_order_id, razorpay_payment_id, razorpay_signature } = req.body;
 const userId = req.user.id;

 // Verify signature
 const sign = razorpay_order_id + '|' + razorpay_payment_id;
 const expectedSign = crypto
 .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
 .update(sign.toString())

```

```

 .digest('hex');

 if (razorpay_signature !== expectedSign) {
 return res.status(400).json({ error: 'Invalid payment signature' });
 }

 // Update payment status
 await pool.query(
 'UPDATE payments SET razorpay_payment_id = $1, razorpay_signature = $2, status = $3 WHERE razorpay_order_id = $4',
 [razorpay_payment_id, razorpay_signature, 'completed', razorpay_order_id]
);

 // Upgrade user to paid
 await pool.query(
 'UPDATE users SET is_paid = true WHERE id = $1',
 [userId]
);

 res.json({
 message: 'Payment verified successfully',
 success: true
 });
 } catch (error) {
 console.error('Verify payment error:', error);
 res.status(500).json({ error: 'Payment verification failed' });
 }
};

const getPaymentStatus = async (req, res) => {
 try {
 const userId = req.user.id;

 const result = await pool.query(
 'SELECT * FROM payments WHERE user_id = $1 ORDER BY created_at DESC LIMIT 1',
 [userId]
);

 if (result.rows.length === 0) {
 return res.json({ hasPaid: false });
 }

 const payment = result.rows[0];
 res.json({
 hasPaid: payment.status === 'completed',
 payment: payment
 });
 } catch (error) {
 console.error('Get payment status error:', error);
 res.status(500).json({ error: 'Failed to get payment status' });
 }
};

module.exports = {
 createOrder,
 verifyPayment,

```

```

 getPaymentStatus
 };
 ...

 ### `backend/routes/auth.js`
  ```javascript
  const express = require('express');
  const router = express.Router();
  const passport = require('passport');
  const { body } = require('express-validator');
  const { signup, login, googleAuthCallback, getCurrentUser } = require('../controllers/authController');
  const { authenticate } = require('../middleware/auth');

  // Email/Password signup
  router.post('/signup', [
    body('email').isEmail().normalizeEmail(),
    body('password').isLength({ min: 6 })
  ], signup);

  // Email/Password login
  router.post('/login', [
    body('email').isEmail().normalizeEmail(),
    body('password').notEmpty()
  ], login);

  // Google OAuth routes
  router.get('/google',
    passport.authenticate('google', {
      scope: ['profile', 'email'],
      session: false
    })
  );

  router.get('/google/callback',
    passport.authenticate('google', {
      failureRedirect: `${process.env.FRONTEND_URL}/login`,
      session: false
    }),
    googleAuthCallback
  );

  // Get current user
  router.get('/me', authenticate, getCurrentUser);

  module.exports = router;
  ...

  ### `backend/routes/files.js`
  ```javascript
 const express = require('express');
 const router = express.Router();
 const upload = require('../middleware/upload');
 const { authenticate, checkUploadLimit } = require('../middleware/auth');
 const { uploadFile, getUserFiles, deleteFile } = require('../controllers/fileController');

 // Upload file

```

```

router.post('/upload', authenticate, checkUploadLimit, upload.single('file'), uploadFile);

// Get user files
router.get('/', authenticate, getUserFiles);

// Delete file
router.delete('/:id', authenticate, deleteFile);

module.exports = router;
...

`backend/routes/search.js`
```javascript
const express = require('express');
const router = express.Router();
const { authenticate } = require('../middleware/auth');
const { searchFiles } = require('../controllers/searchController');

// Search files
router.post('/', authenticate, searchFiles);

module.exports = router;
...

### `backend/routes/payment.js`
```javascript
const express = require('express');
const router = express.Router();
const { authenticate } = require('../middleware/auth');
const { createOrder, verifyPayment, getPaymentStatus } = require('../controllers/paymentController');

// Create payment order
router.post('/create-order', authenticate, createOrder);

// Verify payment
router.post('/verify', authenticate, verifyPayment);

// Get payment status
router.get('/status', authenticate, getPaymentStatus);

module.exports = router;
...

`backend/server.js`
```javascript
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const passport = require('passport');
require('dotenv').config();

const app = express();

// Middleware
app.use(helmet());

```

```

app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(passport.initialize());

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // limit each IP to 100 requests per windowMs
});
app.use('/api/', limiter);

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/files', require('./routes/files'));
app.use('/api/search', require('./routes/search'));
app.use('/api/payment', require('./routes/payment'));

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

// Error handler
app.use((err, req, res, next) => {
  console.error('Error:', err);
  res.status(err.status || 500).json({
    error: err.message || 'Internal server error'
  });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
  console.log(`🌐 Environment: ${process.env.NODE_ENV || 'development'}`);
});

module.exports = app;
'''

---

## 🧑‍💻 FRONTEND FILES

### `frontend/package.json`
'''json
{

```



```

"name": "memvault-frontend",
"version": "1.0.0",
"private": true,
"dependencies": {
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-router-dom": "^6.20.1",
  "react-scripts": "5.0.1",
  "axios": "^1.6.2",
  "bootstrap": "^5.3.2",
  "react-bootstrap": "^2.9.1"
},
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
"eslintConfig": {
  "extends": [
    "react-app"
  ]
},
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
}
}
...

```

```

### `frontend/.env.example`
``env
REACT_APP_API_URL=http://localhost:5000/api
REACT_APP_RAZORPAY_KEY_ID=your_razorpay_key_id
``

```

```

### `frontend/public/index.html`
``html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="theme-color" content="#000000" />
  <meta name="description" content="Never lose important screenshots and documents again" />
  <title>MemVault - Your Personal Memory Vault</title>
  <script src="https://checkout.razorpay.com/v1/checkout.js"></script>
</head>

```

```

</body>
  <noscript>You need to enable JavaScript to run this app.</noscript>
  <div id="root"></div>
</body>
</html>
'''

```

```

### `frontend/src/index.js`
```javascript
import React from 'react';
import ReactDOM from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import './App.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
 <React.StrictMode>
 <App />
 </React.StrictMode>
);
'''

```

```

`frontend/src/App.js`
```javascript
import React, { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import LandingPage from './components/LandingPage';
import Login from './components/Login';
import Signup from './components/Signup';
import Dashboard from './components/Dashboard';
import PrivateRoute from './components/PrivateRoute';
import api from './services/api';

```

```

function App() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      fetchCurrentUser();
    } else {
      setLoading(false);
    }
  }, []);

```

```

const fetchCurrentUser = async () => {
  try {
    const response = await api.get('/auth/me');
    setUser(response.data.user);
  } catch (error) {
    console.error('Failed to fetch user:', error);
    localStorage.removeItem('token');
  } finally {

```

```

    setLoading(false);
  }
};

const handleLogin = (userData) => {
  setUser(userData);
};

const handleLogout = () => {
  localStorage.removeItem('token');
  setUser(null);
};

if (loading) {
  return (
    <div className="d-flex justify-content-center align-items-center" style={{ minHeight: '100vh' }}>
      <div className="spinner-border text-primary" role="status">
        <span className="visually-hidden">Loading...</span>
      </div>
    </div>
  );
}

return (
  <Router>
    <div className="App">
      <Navbar user={user} onLogout={handleLogout} />
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route
          path="/login"
          element={user ? <Navigate to="/dashboard" /> : <Login onLogin={handleLogin} />}
        />
        <Route
          path="/signup"
          element={user ? <Navigate to="/dashboard" /> : <Signup onLogin={handleLogin} />}
        />
        <Route
          path="/dashboard"
          element={
            <PrivateRoute user={user}>
              <Dashboard user={user} setUser={setUser} />
            </PrivateRoute>
          }
        />
        <Route
          path="/auth/google/callback"
          element={<GoogleCallback onLogin={handleLogin} />}
        />
      </Routes>
    </div>
  </Router>
);
}

const GoogleCallback = ({ onLogin }) => {

```

```

useEffect(() => {
  const params = new URLSearchParams(window.location.search);
  const token = params.get('token');

  if (token) {
    localStorage.setItem('token', token);

    api.get('/auth/me')
      .then(response => {
        onLogin(response.data.user);
        window.location.href = '/dashboard';
      })
      .catch(error => {
        console.error('Authentication failed:', error);
        window.location.href = '/login?error=auth_failed';
      });
  } else {
    window.location.href = '/login?error=no_token';
  }
}, [onLogin]);

return (
  <div className="d-flex justify-content-center align-items-center" style={{ minHeight: '100vh' }}>
    <div className="spinner-border text-primary" role="status">
      <span className="visually-hidden">Authenticating...</span>
    </div>
  </div>
);
};

export default App;
'''

#### `frontend/src/App.css`
'''css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  background-color: #f8f9fa;
}

.App {
  min-height: 100vh;
}

/* Landing Page */
.landing-hero {

```

```
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
padding: 100px 0 80px;
text-align: center;
}
```

```
.landing-hero h1 {
font-size: 3rem;
font-weight: 700;
margin-bottom: 1.5rem;
}
```

```
.landing-hero p {
font-size: 1.25rem;
margin-bottom: 2rem;
opacity: 0.95;
}
```

```
.feature-section {
padding: 80px 0;
}
```

```
.feature-card {
border: none;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
transition: transform 0.3s ease;
height: 100%;
}
```

```
.feature-card:hover {
transform: translateY(-5px);
}
```

```
.feature-icon {
font-size: 3rem;
margin-bottom: 1rem;
}
```

```
/* Dashboard */
.dashboard-container {
padding: 40px 0;
min-height: calc(100vh - 56px);
}
```

```
.upload-section {
background: white;
border-radius: 12px;
padding: 30px;
box-shadow: 0 2px 12px rgba(0, 0, 0, 0.08);
margin-bottom: 30px;
}
```

```
.upload-dropzone {
border: 3px dashed #dee2e6;
border-radius: 12px;
padding: 50px 20px;
```

```

text-align: center;
cursor: pointer;
transition: all 0.3s ease;
background-color: #f8f9fa;
}

.upload-dropzone:hover {
border-color: #667eea;
background-color: #f0f2ff;
}

.upload-dropzone.dragging {
border-color: #667eea;
background-color: #e8ebff;
}

.file-input {
display: none;
}

.usage-indicator {
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
padding: 15px 20px;
border-radius: 8px;
margin-bottom: 20px;
display: flex;
justify-content: space-between;
align-items: center;
}

.search-section {
margin-bottom: 30px;
}

.search-bar-container {
position: relative;
}

.search-bar-container input {
padding-left: 45px;
height: 50px;
border-radius: 25px;
border: 2px solid #dee2e6;
font-size: 1rem;
}

.search-bar-container input:focus {
border-color: #667eea;
box-shadow: 0 0 0 0.2rem rgba(102, 126, 234, 0.25);
}

.search-icon {
position: absolute;
left: 15px;
top: 50%;

```

```

transform: translateY(-50%);
font-size: 1.25rem;
color: #6c757d;
}

.files-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
gap: 20px;
margin-top: 30px;
}

.file-card {
background: white;
border-radius: 12px;
overflow: hidden;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
transition: transform 0.2s ease, box-shadow 0.2s ease;
cursor: pointer;
}

.file-card:hover {
transform: translateY(-3px);
box-shadow: 0 4px 16px rgba(0, 0, 0, 0.15);
}

.file-thumbnail {
width: 100%;
height: 200px;
object-fit: cover;
background-color: #f8f9fa;
}

.file-icon-container {
width: 100%;
height: 200px;
display: flex;
align-items: center;
justify-content: center;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
font-size: 4rem;
}

.file-info {
padding: 15px;
}

.file-name {
font-weight: 600;
font-size: 0.95rem;
margin-bottom: 8px;
overflow: hidden;
text-overflow: ellipsis;
white-space: nowrap;
}

```

```
.file-date {
  font-size: 0.85rem;
  color: #6c757d;
  margin-bottom: 10px;
}

.file-keywords {
  display: flex;
  flex-wrap: wrap;
  gap: 5px;
  margin-top: 10px;
}

.keyword-badge {
  background-color: #e8ebff;
  color: #667eea;
  padding: 4px 10px;
  border-radius: 12px;
  font-size: 0.8rem;
  font-weight: 500;
}

.delete-btn {
  position: absolute;
  top: 10px;
  right: 10px;
  background: rgba(255, 255, 255, 0.9);
  border: none;
  border-radius: 50%;
  width: 32px;
  height: 32px;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  transition: all 0.2s ease;
}

.delete-btn:hover {
  background: #dc3545;
  color: white;
}

.empty-state {
  text-align: center;
  padding: 60px 20px;
  color: #6c757d;
}

.empty-state-icon {
  font-size: 4rem;
  margin-bottom: 20px;
  opacity: 0.5;
}
```

```

/* Auth Pages */
.auth-container {
  min-height: calc(100vh - 56px);
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 40px 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

```

```

.auth-card {
  background: white;
  border-radius: 16px;
  padding: 40px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
  width: 100%;
  max-width: 450px;
}

```

```

.auth-card h2 {
  margin-bottom: 30px;
  text-align: center;
  color: #333;
}

```

```

.google-btn {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 12px;
  border: 2px solid #dee2e6;
  background: white;
  border-radius: 8px;
  font-weight: 500;
  transition: all 0.2s ease;
}

```

```

.google-btn:hover {
  background: #f8f9fa;
  border-color: #667eea;
}

```

```

.divider {
  display: flex;
  align-items: center;
  text-align: center;
  margin: 25px 0;
  color: #6c757d;
}

```

```

.divider::before,
.divider::after {
  content: '';
  flex: 1;
}

```

```

border-bottom: 1px solid #dee2e6;
}

.divider span {
padding: 0 15px;
}

/* Paywall Modal */
.modal-content {
border-radius: 16px;
border: none;
}

.modal-header {
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
border-radius: 16px 16px 0 0;
padding: 25px;
}

.price-display {
text-align: center;
padding: 30px;
background: #f8f9fa;
border-radius: 12px;
margin: 20px 0;
}

.price-amount {
font-size: 3rem;
font-weight: 700;
color: #667eea;
}

.price-period {
font-size: 1.25rem;
color: #6c757d;
}

/* Responsive */
@media (max-width: 768px) {
.landing-hero h1 {
font-size: 2rem;
}

.landing-hero p {
font-size: 1rem;
}

.files-grid {
grid-template-columns: 1fr;
}

.auth-card {
padding: 30px 20px;
}

```

```

}

/* Loading States */
.spinner-border-sm {
  width: 1rem;
  height: 1rem;
  border-width: 0.15em;
}

/* Utility Classes */
.text-gradient {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.btn-gradient {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border: none;
  color: white;
  transition: transform 0.2s ease;
}

.btn-gradient:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(102, 126, 234, 0.4);
  color: white;
}
...

### `frontend/src/services/api.js`
```javascript
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const api = axios.create({
 baseURL: API_URL,
 headers: {
 'Content-Type': 'application/json'
 }
});

// Add token to requests
api.interceptors.request.use(
 (config) => {
 const token = localStorage.getItem('token');
 if (token) {
 config.headers.Authorization = `Bearer ${token}`;
 }
 return config;
 },
 (error) => {
 return Promise.reject(error);
 }
)

```

```

);

// Handle responses
api.interceptors.response.use(
 (response) => response,
 (error) => {
 if (error.response && error.response.status === 401) {
 localStorage.removeItem('token');
 window.location.href = '/login';
 }
 return Promise.reject(error);
 }
);

export default api;
'''

`frontend/src/components/Navbar.js`
'''javascript
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { Navbar as BootstrapNavbar, Nav, Container, Button } from 'react-bootstrap';

function Navbar({ user, onLogout }) {
 const navigate = useNavigate();

 const handleLogout = () => {
 onLogout();
 navigate('/');
 };

 return (
 <BootstrapNavbar bg="white" expand="lg" className="shadow-sm">
 <Container>
 <BootstrapNavbar.Brand as={Link} to="/" className="fw-bold text-gradient">
 📦 MemVault
 </BootstrapNavbar.Brand>
 <BootstrapNavbar.Toggle aria-controls="basic-navbar-nav" />
 <BootstrapNavbar.Collapse id="basic-navbar-nav">
 <Nav className="ms-auto align-items-center">
 {user ? (
 <>
 <Nav.Link as={Link} to="/dashboard">
 Dashboard
 </Nav.Link>

 {user.email}

 <Button variant="outline-danger" size="sm" onClick={handleLogout}>
 Logout
 </Button>
 </>
) : (
 <>
 <Nav.Link as={Link} to="/login">
 Login

```

```

 </Nav.Link>
 <Button as={Link} to="/signup" variant="primary" size="sm" className="ms-2 btn-gradient">
 Sign

```

Continue

4:42 pm

Up </Button> </>

)} </Nav> </BootstrapNavbar.Collapse> </Container> </BootstrapNavbar> ); }

export default Navbar;

```

`frontend/src/components/LandingPage.js`
```javascript
import React from 'react';
import { Container, Row, Col, Card, Button } from 'react-bootstrap';
import { Link } from 'react-router-dom';

function LandingPage() {
  return (
    <div>
      { /* Hero Section */ }
      <div className="landing-hero">
        <Container>
          <h1>Never lose important screenshots and documents again</h1>
          <p className="lead">
            Your personal memory vault for screenshots, PDFs, and documents.<br />
            Search your files using natural language.
          </p>
          <Button as={Link} to="/signup" size="lg" variant="light" className="mt-3">
            Get Started Free - 5 Uploads
          </Button>
        </Container>
      </div>

      { /* Features Section */ }
      <Container className="feature-section">
        <h2 className="text-center mb-5">How MemVault Works</h2>
        <Row>
          <Col md={4} className="mb-4">
            <Card className="feature-card text-center p-4">
              <div className="feature-icon">📄</div>
              <Card.Body>
                <Card.Title>Upload Your Files</Card.Title>
                <Card.Text>
                  Save important screenshots, PDFs, and DOCX files to your private vault.
                  Supported formats: JPG, PNG, PDF, DOCX.
                </Card.Text>
              </Card.Body>
            </Card>
          </Col>
          <Col md={4} className="mb-4">
            <Card className="feature-card text-center p-4">
              <div className="feature-icon">🔍</div>

```

```

    <Card.Body>
    <Card.Title>Search Naturally</Card.Title>
    <Card.Text>
    Find your files instantly using everyday language.
    Search for "Amazon refund" or "rent agreement" and get results.
    </Card.Text>
    </Card.Body>
  </Card>
</Col>
<Col md={4} className="mb-4">
  <Card className="feature-card text-center p-4">
    <div className="feature-icon">🔒 </div>
    <Card.Body>
    <Card.Title>Private & Secure</Card.Title>
    <Card.Text>
    Your files are stored securely in the cloud.
    No automatic syncing. No spying. You upload only what you want.
    </Card.Text>
    </Card.Body>
  </Card>
</Col>
</Row>
</Container>

```

```

{/* Pricing Section */}
<div style={{ background: '#f8f9fa', padding: '80px 0' }}>
  <Container>
    <h2 className="text-center mb-5">Simple, Affordable Pricing</h2>
    <Row className="justify-content-center">
      <Col md={8} lg={6}>
        <Card className="text-center p-4 shadow">
          <Card.Body>
            <h3 className="mb-4">Start Free, Upgrade Anytime</h3>
            <div className="mb-4">
              <div className="h1 text-gradient">5 Free Uploads</div>
              <p className="text-muted">Try MemVault with no credit card required</p>
            </div>
            <hr />
            <div className="mt-4">
              <div className="h2 text-gradient">₹99/year</div>
              <p className="text-muted mb-3">For users in India</p>
              <div className="h2 text-gradient">$15/year</div>
              <p className="text-muted">For international users</p>
            </div>
            <div className="mt-4">
              <p className="fw-bold">Unlimited uploads and searches</p>
            </div>
            <Button as={Link} to="/signup" size="lg" className="mt-3 btn-gradient">
              Get Started Free
            </Button>
          </Card.Body>
        </Card>
      </Col>
    </Row>
  </Container>
</div>

```

```

    { /* Privacy Section */ }
    <Container className="py-5">
      <Row className="justify-content-center">
        <Col md={8} className="text-center">
          <h3 className="mb-4">Privacy First</h3>
          <p className="lead">
            MemVault is not a gallery replacement or automatic backup tool.
            You deliberately choose what to save. Your files are encrypted and stored
            securely. We never access or use your personal documents.
          </p>
        </Col>
      </Row>
    </Container>
  </div>
);
}

export default LandingPage;
'''

### `frontend/src/components/Login.js`
'''javascript
import React, { useState } from 'react';
import { Form, Button, Alert, Container } from 'react-bootstrap';
import { Link } from 'react-router-dom';
import api from '../services/api';

function Login({ onLogin }) {
  const [formData, setFormData] = useState({
    email: '',
    password: ''
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      const response = await api.post('/auth/login', formData);
      localStorage.setItem('token', response.data.token);
      onLogin(response.data.user);
      window.location.href = '/dashboard';
    } catch (err) {
      setError(err.response?.data?.error || 'Login failed');
    } finally {

```

```

    setLoading(false);
  }
};

const handleGoogleLogin = () => {
  window.location.href = `${process.env.REACT_APP_API_URL || 'http://localhost:5000/api'}/auth/google`;
};

return (
  <div className="auth-container">
    <Container>
      <div className="auth-card mx-auto">
        <h2>Welcome Back</h2>
        {error && <Alert variant="danger">{error}</Alert>}
        <Button
          variant="outline-secondary"
          className="google-btn"
          onClick={handleGoogleLogin}
        >
          <span><img alt="Google logo" data-bbox="218 398 235 412"/></span> Continue with Google
        </Button>
      </div>
      <div className="divider">
        <span>or</span>
      </div>
      <Form onSubmit={handleSubmit}>
        <Form.Group className="mb-3">
          <Form.Label>Email</Form.Label>
          <Form.Control
            type="email"
            name="email"
            value={formData.email}
            onChange={handleChange}
            required
            placeholder="your@email.com"
          />
        </Form.Group>
        <Form.Group className="mb-3">
          <Form.Label>Password</Form.Label>
          <Form.Control
            type="password"
            name="password"
            value={formData.password}
            onChange={handleChange}
            required
            placeholder="••••••••"
            minLength={6}
          />
        </Form.Group>
        <Button
          variant="primary"

```



```

        type="submit"
        className="w-100 btn-gradient"
        disabled={loading}
      >
        {loading ? 'Logging in...' : 'Login'}
      </Button>
    </Form>

    <div className="text-center mt-3">
      <p>
        Don't have an account? <Link to="/signup">Sign up</Link>
      </p>
    </div>
  </div>
</Container>
</div>
);
}

export default Login;
```

`frontend/src/components/Signup.js`
```javascript
import React, { useState } from 'react';
import { Form, Button, Alert, Container } from 'react-bootstrap';
import { Link } from 'react-router-dom';
import api from '../services/api';

function Signup({ onLogin }) {
  const [formData, setFormData] = useState({
    email: '',
    password: '',
    confirmPassword: '',
    country: 'IN'
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    if (formData.password !== formData.confirmPassword) {
      setError('Passwords do not match');
      return;
    }

    setLoading(true);

```

```

    try {
      const response = await api.post('/auth/signup', {
        email: formData.email,
        password: formData.password,
        country: formData.country
      });
      localStorage.setItem('token', response.data.token);
      onLogin(response.data.user);
      window.location.href = '/dashboard';
    } catch (err) {
      setError(err.response?.data?.error || 'Signup failed');
    } finally {
      setLoading(false);
    }
  };

  const handleGoogleSignup = () => {
    window.location.href = `${process.env.REACT_APP_API_URL || 'http://localhost:5000/api'}/auth/google`;
  };

  return (
    <div className="auth-container">
      <Container>
        <div className="auth-card mx-auto">
          <h2>Create Account</h2>
          {error && <Alert variant="danger">{error}</Alert>}
          <Button
            variant="outline-secondary"
            className="google-btn"
            onClick={handleGoogleSignup}
          >
            <span><img alt="Google logo" data-bbox="218 588 235 605"/> Sign up with Google</span>
          </Button>
          <div className="divider">
            <span>or</span>
          </div>
          <Form onSubmit={handleSubmit}>
            <Form.Group className="mb-3">
              <Form.Label>Email</Form.Label>
              <Form.Control
                type="email"
                name="email"
                value={formData.email}
                onChange={handleChange}
                required
                placeholder="your@email.com"
              />
            </Form.Group>
            <Form.Group className="mb-3">
              <Form.Label>Password</Form.Label>

```

```

<Form.Control
  type="password"
  name="password"
  value={formData.password}
  onChange={handleChange}
  required
  placeholder="••••••••"
  minLength={6}
/>
<Form.Text className="text-muted">
  Minimum 6 characters
</Form.Text>
</Form.Group>

```

```

<Form.Group className="mb-3">
  <Form.Label>Confirm Password</Form.Label>
  <Form.Control
    type="password"
    name="confirmPassword"
    value={formData.confirmPassword}
    onChange={handleChange}
    required
    placeholder="••••••••"
  />
</Form.Group>

```

```

<Form.Group className="mb-3">
  <Form.Label>Country</Form.Label>
  <Form.Select
    name="country"
    value={formData.country}
    onChange={handleChange}
  >
    <option value="IN">India</option>
    <option value="US">United States</option>
    <option value="GB">United Kingdom</option>
    <option value="OTHER">Other</option>
  </Form.Select>
  <Form.Text className="text-muted">
    Required for pricing (₹99/year in India, $15/year elsewhere)
  </Form.Text>
</Form.Group>

```

```

<Button
  variant="primary"
  type="submit"
  className="w-100 btn-gradient"
  disabled={loading}
>
  {loading ? 'Creating account...' : 'Sign Up'}
</Button>
</Form>

```

```

<div className="text-center mt-3">
  <p>
    Already have an account? <Link to="/login">Login</Link>
  </p>

```

```

        </p>
      </div>
    </div>
  </Container>
</div>
);
}

export default Signup;
.....

#### `frontend/src/components/Dashboard.js`
```javascript
import React, { useState, useEffect } from 'react';
import { Container, Alert } from 'react-bootstrap';
import UploadSection from './UploadSection';
import SearchBar from './SearchBar';
import FileCard from './FileCard';
import PaywallModal from './PaywallModal';
import api from '../services/api';

function Dashboard({ user, setUser }) {
 const [files, setFiles] = useState([]);
 const [filteredFiles, setFilteredFiles] = useState([]);
 const [loading, setLoading] = useState(true);
 const [showPaywall, setShowPaywall] = useState(false);
 const [uploadError, setUploadError] = useState('');
 const [searchQuery, setSearchQuery] = useState('');
 const [searching, setSearching] = useState(false);

 useEffect(() => {
 fetchFiles();
 }, []);

 const fetchFiles = async () => {
 try {
 const response = await api.get('/files');
 setFiles(response.data.files);
 setFilteredFiles(response.data.files);
 } catch (error) {
 console.error('Failed to fetch files:', error);
 } finally {
 setLoading(false);
 }
 };

 const handleFileUploaded = (newFile) => {
 setFiles([newFile, ...files]);
 setFilteredFiles([newFile, ...filteredFiles]);
 setUploadError('');

 // Update user data
 if (user && !user.is_paid) {
 setUser({
 ...user,
 free_uploads_used: user.free_uploads_used + 1
 });
 }
 };
}

```

```

 });
 }
};

const handleUploadError = (error) => {
 if (error.requiresPayment) {
 setShowPaywall(true);
 } else {
 setUploadError(error.message);
 }
};

const handleSearch = async (query) => {
 setSearchQuery(query);

 if (!query || query.trim() === '') {
 setFilteredFiles(files);
 return;
 }

 setSearching(true);
 try {
 const response = await api.post('/search', { query });
 setFilteredFiles(response.data.results);
 } catch (error) {
 console.error('Search failed:', error);
 setUploadError('Search failed. Please try again.');
```

} finally {
 setSearching(false);
 }
 }
};

const handleDeleteFile = async (fileId) => {
 try {
 await api.delete(`/files/\${fileId}`);
 setFiles(files.filter(f => f.id !== fileId));
 setFilteredFiles(filteredFiles.filter(f => f.id !== fileId));
 } catch (error) {
 console.error('Failed to delete file:', error);
 setUploadError('Failed to delete file');
 }
};

const handlePaymentSuccess = () => {
 setShowPaywall(false);
 setUser({ ...user, is\_paid: true });
 setUploadError('');
};

const remainingUploads = user.is\_paid ? 'Unlimited' : `\${5 - user.free\_uploads\_used} / 5`;

if (loading) {
 return (
 <Container className="dashboard-container text-center">
 <div className="spinner-border text-primary" role="status">
 <span className="visually-hidden">Loading...</span>
 </div>
 </Container>
 );
}

```

 </div>
 </Container>
);
}

return (
 <Container className="dashboard-container">
 <div className="usage-indicator">
 <div>
 Uploads Remaining:
 </div>
 <div className="fs-5">{remainingUploads}</div>
 </div>

 {uploadError && (
 <Alert variant="danger" dismissible onClose={() => setUploadError('')}>
 {uploadError}
 </Alert>
)}

 <UploadSection
 onFileUploaded={handleFileUploaded}
 onUploadError={handleUploadError}
 isPaid={user.is_paid}
 />

 <SearchBar
 onSearch={handleSearch}
 searching={searching}
 />

 {filteredFiles.length === 0 ? (
 <div className="empty-state">
 <div className="empty-state-icon"></div>
 <h3>{searchQuery ? 'No files found' : 'No files uploaded yet'}</h3>
 <p className="text-muted">
 {searchQuery
 ? 'Try a different search query'
 : 'Upload your first screenshot or document to get started'}
 </p>
 </div>
) : (
 <div className="files-grid">
 {filteredFiles.map(file => (
 <FileCard
 key={file.id}
 file={file}
 onDelete={handleDeleteFile}
 />
))}
 </div>
)}

 <PaywallModal
 show={showPaywall}
 onHide={() => setShowPaywall(false)}
 />
 </Container>
);

```

```

 userCountry={user.country}
 onPaymentSuccess={handlePaymentSuccess}
 />
 </Container>
);
}

export default Dashboard;
````

### `frontend/src/components/UploadSection.js`
````javascript
import React, { useState, useRef } from 'react';
import { Button } from 'react-bootstrap';
import api from '../services/api';

function UploadSection({ onFileUploaded, onUploadError, isPaid }) {
 const [uploading, setUploading] = useState(false);
 const [dragging, setDragging] = useState(false);
 const fileInputRef = useRef(null);

 const handleFileSelect = async (file) => {
 if (!file) return;

 // Validate file type
 const allowedTypes = ['image/jpeg', 'image/png', 'image/jpg', 'application/pdf',
 'application/vnd.openxmlformats-officedocument.wordprocessingml.document'];

 if (!allowedTypes.includes(file.type)) {
 onUploadError({ message: 'Invalid file type. Only JPG, PNG, PDF, and DOCX files are allowed.' });
 return;
 }

 // Validate file size (10MB)
 if (file.size > 10 * 1024 * 1024) {
 onUploadError({ message: 'File size must be less than 10MB' });
 return;
 }

 setUploading(true);

 const formData = new FormData();
 formData.append('file', file);

 try {
 const response = await api.post('/files/upload', formData, {
 headers: {
 'Content-Type': 'multipart/form-data'
 }
 });
 } catch (error) {
 if (error.response?.data?.requiresPayment) {
 onUploadError({
 requiresPayment: true,

```

```

 message: error.response.data.message
 });
 } else {
 onUploadError({ message: error.response?.data?.error || 'Upload failed' });
 }
 } finally {
 setUploading(false);
 if (fileInputRef.current) {
 fileInputRef.current.value = '';
 }
 }
};

const handleDrop = (e) => {
 e.preventDefault();
 setDragging(false);

 const file = e.dataTransfer.files[0];
 handleFileSelect(file);
};

const handleDragOver = (e) => {
 e.preventDefault();
 setDragging(true);
};

const handleDragLeave = () => {
 setDragging(false);
};

return (
 <div className="upload-section">
 <h4 className="mb-3">Upload Files</h4>
 <div
 className={`upload-dropzone ${dragging ? 'dragging' : ''}`}
 onDrop={handleDrop}
 onDragOver={handleDragOver}
 onDragLeave={handleDragLeave}
 onClick={() => fileInputRef.current?.click()}
 >
 <div className="fs-1 mb-3">📁</div>
 <h5>Drop your file here or click to browse</h5>
 <p className="text-muted mb-3">
 Supported: JPG, PNG, PDF, DOCX (Max 10MB)
 </p>
 {uploading ? (
 <div className="spinner-border text-primary" role="status">
 Uploading...
 </div>
) : (
 <Button variant="primary" className="btn-gradient">
 Choose File
 </Button>
)}
 </div>
 </div>

```



```

 </div>

 <input
 ref={fileInputRef}
 type="file"
 className="file-input"
 accept=".jpg,.jpeg,.png,.pdf,.docx"
 onChange={(e) => handleFileSelect(e.target.files[0])}
 disabled={uploading}
 />
 </div>
);
}

export default UploadSection;
````

#### `frontend/src/components/SearchBar.js`
````javascript
import React, { useState } from 'react';
import { Form, InputGroup } from 'react-bootstrap';

function SearchBar({ onSearch, searching }) {
 const [query, setQuery] = useState("");

 const handleSubmit = (e) => {
 e.preventDefault();
 onSearch(query);
 };

 const handleChange = (e) => {
 const value = e.target.value;
 setQuery(value);

 // Auto-search on empty (to reset)
 if (value === '') {
 onSearch("");
 }
 };

 return (
 <div className="search-section">
 <Form onSubmit={handleSubmit}>
 <div className="search-bar-container">
 🔍
 <Form.Control
 type="text"
 placeholder="Search your screenshots and documents..."
 value={query}
 onChange={handleChange}
 disabled={searching}
 />
 </div>
 </Form>
 {searching && (
 <div className="text-center mt-2">

```

```

 <small className="text-muted">Searching...</small>
 </div>
)}
 </div>
);
}

export default SearchBar;
````

#### `frontend/src/components/FileCard.js`
````javascript
import React, { useState } from 'react';

function FileCard({ file, onDelete }) {
 const [showDelete, setShowDelete] = useState(false);

 const formatDate = (dateString) => {
 const date = new Date(dateString);
 return date.toLocaleDateString('en-US', {
 year: 'numeric',
 month: 'short',
 day: 'numeric'
 });
 };

 const isImage = file.file_type.startsWith('image/');
 const isPDF = file.file_type === 'application/pdf';
 const isDOCX = file.file_type === 'application/vnd.openxmlformats-officedocument.wordprocessingml.document';

 const handleDelete = (e) => {
 e.stopPropagation();
 if (window.confirm('Are you sure you want to delete this file?')) {
 onDelete(file.id);
 }
 };

 const handleClick = () => {
 window.open(file.file_url, '_blank');
 };

 return (
 <div
 className="file-card position-relative"
 onMouseEnter={() => setShowDelete(true)}
 onMouseLeave={() => setShowDelete(false)}
 onClick={handleClick}
 >
 {showDelete && (
 <button
 className="delete-btn"
 onClick={handleDelete}
 title="Delete file"
 >


```

```

 </button>
)}

 {isImage ? (
 <img
 src={file.file_url}
 alt={file.file_name}
 className="file-thumbnail"
 />
) : (
 <div className="file-icon-container">
 {isPDF ? '📄' : isDOCX ? '📝' : '📋'}
 </div>
)}

 <div className="file-info">
 <div className="file-name" title={file.file_name}>
 {file.file_name}
 </div>
 <div className="file-date">
 {formatDate(file.created_at)}
 </div>

 {file.keywords && file.keywords.length > 0 && (
 <>
 <div className="text-muted small mt-2">Detected keywords:</div>
 <div className="file-keywords">
 {file.keywords.map((keyword, index) => (

 {keyword}

))}
 </div>
 </>
)}

 {file.preview && (
 <div className="text-muted small mt-2" style={{ fontSize: '0.75rem' }}>
 {file.preview}...
 </div>
)}
 </div>
);
}

export default FileCard;
````

### `frontend/src/components/PaywallModal.js`
````javascript
import React, { useState } from 'react';
import { Modal, Button, Alert } from 'react-bootstrap';
import api from '../services/api';

function PaywallModal({ show, onHide, userCountry, onPaymentSuccess }) {

```

```

const [loading, setLoading] = useState(false);
const [error, setError] = useState('');

const isIndia = userCountry === 'IN';
const price = isIndia ? '₹99' : '$15';
const currency = isIndia ? 'INR' : 'USD';

const handlePayment = async () => {
 setLoading(true);
 setError('');

 try {
 // Create order
 const orderResponse = await api.post('/payment/create-order');
 const { orderId, amount, currency, keyId } = orderResponse.data;

 // Initialize Razorpay
 const options = {
 key: keyId,
 amount: amount,
 currency: currency,
 name: 'MemVault',
 description: 'Annual Subscription',
 order_id: orderId,
 handler: async function (response) {
 try {
 // Verify payment
 const verifyResponse = await api.post('/payment/verify', {
 razorpay_order_id: response.razorpay_order_id,
 razorpay_payment_id: response.razorpay_payment_id,
 razorpay_signature: response.razorpay_signature
 });

 if (verifyResponse.data.success) {
 onPaymentSuccess();
 alert('Payment successful! You now have unlimited uploads.');
 }
 } catch (err) {
 setError('Payment verification failed');
 }
 },
 prefill: {
 email: ''
 },
 theme: {
 color: '#667eea'
 }
 };

 const razorpay = new window.Razorpay(options);
 razorpay.on('payment.failed', function () {
 setError('Payment failed. Please try again.');
 });
 razorpay.open();
 } catch (err) {
 setError('Failed to initiate payment');
 }
}

```

```

 } finally {
 setLoading(false);
 }
 };

 return (
 <Modal show={show} onHide={onHide} centered>
 <Modal.Header closeButton>
 <Modal.Title>Upgrade to Unlimited</Modal.Title>
 </Modal.Header>
 <Modal.Body>
 {error && <Alert variant="danger">{error}</Alert>}

 <div className="text-center mb-4">
 <p className="lead">
 You've used all 5 free uploads!
 </p>
 <p>
 Upgrade now to get unlimited uploads and searches.
 </p>
 </div>

 <div className="price-display">
 <div className="price-amount">{price}</div>
 <div className="price-period">/year</div>
 </div>

 <ul className="list-unstyled mt-4">
 <li className="mb-2">✔ Unlimited file uploads
 <li className="mb-2">✔ Unlimited searches
 <li className="mb-2">✔ Secure cloud storage
 <li className="mb-2">✔ Priority support

 </Modal.Body>
 <Modal.Footer>
 <Button variant="secondary" onClick={onHide}>
 Maybe Later
 </Button>
 <Button
 variant="primary"
 className="btn-gradient"
 onClick={handlePayment}
 disabled={loading}
 >
 {loading ? 'Processing...' : `Pay ${price}`}
 </Button>
 </Modal.Footer>
 </Modal>
);
}

export default PaywallModal;
````

#### `frontend/src/components/PrivateRoute.js`
````javascript

```

```
import React from 'react';
import { Navigate } from 'react-router-dom';
```

```
function PrivateRoute({ user, children }) {
 if (!user) {
 return <Navigate to="/login" replace />;
 }
}
```

```
 return children;
}
```

```
export default PrivateRoute;
````
```

```
---
```

```
## 📖 README.md
```

```
````markdown
```

```
MemVault - Personal Memory Vault SaaS
```

A production-ready SaaS application for storing and searching important screenshots, PDFs, and documents using semantic search.

## ## Features

- Upload images (JPG, PNG), PDFs, and DOCX files
- Automatic text extraction (OCR + document parsing)
- Semantic search using vector embeddings
- User authentication (Email/Password + Google OAuth)
- Freemium model (5 free uploads, then paid)
- Payment processing with Razorpay
- Country-based pricing
- Secure cloud storage with Cloudinary

## ## Tech Stack

### ### Backend

- Node.js + Express
- PostgreSQL + pgvector
- Tesseract.js (OCR)
- pdf-parse & mammoth (document parsing)
- OpenAI Embeddings API
- Razorpay (payments)
- Cloudinary (file storage)
- Passport.js (authentication)

### ### Frontend

- React
- React Router
- Bootstrap
- Axios

## ## Prerequisites

1. Node.js (v16+)
2. PostgreSQL (v12+)

3. Cloudinary account
4. OpenAI API key
5. Razorpay account
6. Google OAuth credentials (optional)

## ## Installation

### ### 1. Clone Repository

```
```bash
git clone <repository-url>
cd memvault
```
```

### ### 2. Database Setup

```
```bash
# Install PostgreSQL and create database
createdb memvault
```

```
# Enable pgvector extension
psql memvault
CREATE EXTENSION vector
```